

Cloud Practitioner - FreeCodeCamp

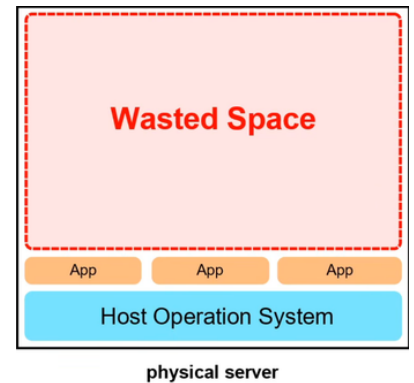
Cloud computing - ใช้ server คนอื่นประมวลผล/เก็บข้อมูลแทนที่จะใช้ของตัวเอง(On-premise)

Common cloud services for IaaS - Compute(EC2), networking(VPS), storage(EBS), databases(RDS)

Computing คลายแบบ

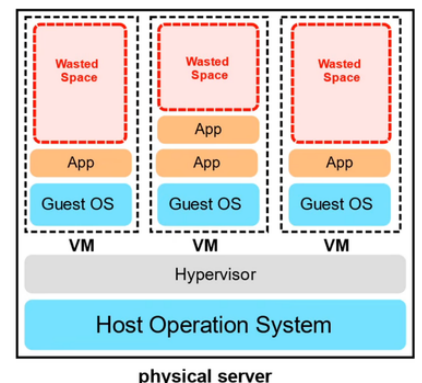
1. Dedicated (Bare Metal)

- **physical server** มาอยู่ที่ตัวเราเลย
- ขาดการแยกกัน **ไม่มี isolation ระหว่าง Application**
- **Vertical scale** (การ upgrade เครื่อง เช่น RAM, CPU) ยาก
- หลาย app อาจตีกันเรื่องแบ่งทรัพยากร (มันต้องจองแย่งกัน) และ environment อาจไม่ตรงกัน (dependency hell)
- อาจใช้ใน High performance computing (**พวก ต้องการ OVERHEAD ต่ำๆ**)



2. VMs (machine in machine)

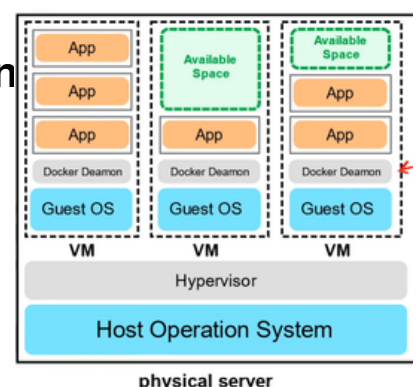
- จะ **generate new OS (guest OS)** ขึ้นมาซ้อนเป็นเหมือนอีกเครื่องโดยมี **Hypervisor** จัดการพวก **instances VMs** (เช่น VirtualBox or AWS Nitro System)
- **Isolate** ได้ระดับนึง (different instance of VM) แต่ใน **VM** นึง **ยังรับ app พร้อมกันไม่ได้ชนกันปัญหาเดิม**
- **Vertical Scale + Horizontal Scale (เพิ่มเครื่อง)** ง่าย
- ก็แค่ไปปรับ config VM
- **ยังเสียทรัพยากรบางส่วนสูญเปล่าใน VM อยู่**
- **เพิ่มเติม overhead ที่เสียไปในการสร้าง VM**



3. Containers

ใน **OS เดียวกัน** เราต้องการให้ **run many app** โดยไม่ชน เราก็เลยมีสิ่งที่เรียกว่า **containers** ในภาพเป็น **Docker Deamon** **ตัวจัดการ container**

- container utilize mem ได้ดีมาก (**Shared Kernel**) เพราะสร้าง instance เพิ่มได้มาใช้
 - เร็ว เพราะ **ไม่ต้องไป load kernel** ใหม่แบบ VM
 - แต่ละ instance container มี dependency ต่างกัน (**complete ISOLATION**) migration ง่ายขึ้นด้วย มีข้อเสียแค่ maintain เยอะ
- ## 4. Functions (Serverless computing ex. AWS LAMBDA)
- **เราแค่โค้ดขึ้นไปรันเลยบอก environment ให้เรียบร้อย ไม่ต้องไปนั่ง maintain** แบบ container
 - **จ่ายแค่ช่วงที่รัน เลือกความจำได้** ค่อนข้างง่ายและสะดวก (**event-driven**)
 - อาจมีปัญหา **Cold-Start** ตอนรอ VM run หรือ request ไปแล้วอาจช้า + **TIME LIMIT**



Type of cloud computing

1. Software as a Service (SaaS) - ให้ software มาใช้เลย

ex. github

2. Platform as a Service (PaaS) - ให้ platform เราใช้แค่การ develop app ได้เลย

ex. AWS Elastic beanstalk , Heroku

3. Infrastructure as a Service (IaaS) - ให้มาเชิ้ระดับ OS ขึ้นไป แต่ไม่ต้องไปยุ่ง HW

ex. AWS Elastic Compute cloud(EC2)

Cloud computing deployment models

Public cloud - ใช้ CSP Shared (Multi-tenant) กับหลายเจ้า คั่นโดยกำแพงนิ่ดเดียว

Private cloud - ใช้ on premise or อาจไปขอใช้ของ CSP แต่ server นี้ต้องไม่ public

Hybrid - ผสมสอง และเชื่อมกัน

Cross cloud - ใช้ Multiple CSPs

Computing power - ภาพรวมความสามารถของระบบคอมพิวเตอร์ในการจัดการงานให้เสร็จ

ตัวที่ใช้ concept นี้ก็ EC2(General Computing) , AWS Inferentia (Inf1) ตัวนี้ใช้ใน

ML/AI , AWS Braket (Quantum Computing) อาจมี AWS graviton (ARM)

การเลือก Region เป็นสิ่งที่สำคัญเพราะ

1. ต้องถูกตามกฎหมายโซนนั้นๆ (ต้องระวังพวกข้อบังคับ)

2. cost ไม่เท่า

3. มี service ไรบ้าง

4. ระยะ / latency ไปหา end user

AWS ก็จะมี regional services และ global services ความต่างคือ global เป็น

services ที่มีอยู่หลาย region อยู่แล้วเช่น S3,cloudfront,Route53,IAM

Availability Zones (AZ) - จุดที่มี data center อาจมี 1 หรือหลายที่ ใน AZ นึง (ใน

region 1 data center จะเชื่อมกันด้วย low latency มากๆ < 10ms)สิ่งนี้ก็คือสิ่งที่บริษัท

จะรันหลายๆตัวเพื่อกันมันล่มอะนะ ถ้า common ก็คือ run บน 3 AZs (มันคือเลขตัวท้ายในตัว

region) EC2 ต้องเกิดใน Subnet และ Subnet จะต้องถูกผูกติดกับ AZ ไ่ AZ หนึ่งเสมอ

Fault tolerance terminologies

Fault domain - zone network ที่ถ้ามีตัวสำคัญระเบิด ผลกระทบก็จะอยู่แค่วงนั้น ฟลถ้า

data center ระเบิดก็ระเบิดแค่ส่วนนั้น

Fault level - รวม fault domain ซึ่งใน fault domain ก็แบ่งเป็นหลาย level อีกตาม

ระดับความรุนแรง

Region = fault level AZ = fault domain

SPOF (Single Point of Failure) - จุดที่พังแล้ว ระเบิดเลย

AWS global network - เชื่อมกันระหว่าง AWS global infrastructure

On ramps (User → AWS) - AWS global Accelerator , AWS S3 Transfer

Acceleration

Off ramps (AWS → User) - Amazon CloudFront

VPC endpoints - รับประกันว่าข้อมูลไม่รั่วสู่ public internet

Point of Presence (PoP) - จุดค้นกลางระหว่าง Users and AWS region มักจะเป็น data centers or hardware

Edge Locations - เป็น datacenter ที่ถือ cache เอาไว้ เพื่อลดระยะทาง end users อารมณ์ proxy server

Regional Edge Locations - ฝ่า L2 cache cache ที่ใหญ่กว่าตัว edge และแลกเปลี่ยนข้อมูลกับ edge locations เพื่อไม่ให้ต้องไปเอาจาก server จริง

PoP จะอยู่ในจุดนัดพบ (IXP) ที่เครือข่ายของ AWS มาเชื่อมต่อกับเครือข่ายอินเทอร์เน็ตสาธารณะ

Tier 1 network - เป็น network ที่ไปหาทุกตัวใน internet ได้ โดยไม่เสียตัง (ฝ่าลูกศรชี้ไปหา NP-Hard/Complete)

ซึ่ง PoP ก็เชื่อมตัวนี้แหละ Tier 1 network ที่ให้บริการโดย Tier 1 transit providers เพื่อส่งข้อมูล

Services ที่ใช้ PoP:

Amazon CloudFront - จะทำให้ web เราไปใช้บริการ nearest edge locations เลือกได้ว่าจะใส่ไรไป ในข้อมูลที่จะให้ cache

Amazon S3 Transfer acceleration - เอา URL ที่พิเศษให้ User upload ลง edge locations ได้ ทำให้ ข้อมูลไปถึง S3 ไวขึ้น (download/upload)

AWS global accelerator - หาเส้นทางซึ่งที่สุดของ end users → web server ตัวมันอยู่ ทำให้ user ผ่าน edge location เพื่อให้เร็วมากๆ เพื่อไป web server ตัวจริง (query/update/API calls)

AWS direct connects - เราเอา ของเราไปเชื่อมกับ AWS ก็คือจะได้ option ในการเลือก ก็คือ 1.Dedicated เหมာ Port เองเลย (1G, 10G, 100G) ยึดช่อง

กับ 2.Hosted: แบ่งเช่าผ่าน Partner เริ่มต้นเบาๆ ได้ (50M - 10G) หารช่อง

Local Zones - data center ท่ามกลางคนหมู่มากมีเพื่อผู้ที่ต้องการ super low latency

AWS Wavelength Zones - edge computings ที่อยู่บน 5G network(Mobile)

เหมือนเอา VM ไปเปิดที่ขอบเสาสัญญาณเพื่อ super low latency

Data Residency - ที่อยู่ข้อมูล ที่ต้องทำตาม Compliance Boundaries และ Data Sovereignty พวกกฎหมายทั้งหลาย

AWS config - เป็นตัวไว้ตรวจการตั้งค่า ซึ่งตั้งกฎได้ว่าจะไรเปลี่ยนแล้วแจ้งเตือน อาจ alert ไม่ก็ auto rollback

AWS outposts - เอา HW ไปตั้งที่เราเลย แล้วก็เชื่อมกับเขา

IAM policies - จะเลือกให้ไปหาใครหรือไม่ไปหาใคร หรือแบบคนเข้าก็ใช้ตัวนี้

AWS ground station - ใช้การสื่อสารดาวเทียมในการ ส่งสาร / ภาพถ่ายอะไรตามแต่

Cloud Architecture Terminologies

Solution Architect - นักออกแบบ การแก้ปัญหา technical โดยใช้หลายระบบ

Cloud Architect - ต่างที่โฟกัสโดยจะแก้ปัญหาโดยใช้ cloud

ข้อต้องพิจารณาเวลาออกแบบ Solution

1.Availability - เซิร์ฟเวอร์จะรันตลอดเวลา

requirement - NO SPOF และมีหลาย AZ รันอยู่ อาจมี ELASTIC LB มาช่วยกระจาย load ได้

2.Scalability - ควรจะสามารถขยายต่อไปได้เรื่อยๆ แบบรวดเร็ว

วัดจากความสามารถในการทำ vertical + horizontal scaling

3.Elasticity - ความยืดหยุ่น solution

วัดจากการยืดหด server รับตามload เช่น Horizontal scale out/in ตัวที่ใช้ได้ คือ ASG auto scaling groups

4.Fault tolerance - กันระเบิดแค่ไหน

NO SPOF และถ้าระเบิดก็ไปรัน backup(shift traffic) สามารถใช้ RDS Multi-AZ ได้

5.Disaster recovery - ระเบิดแล้วฟื้นตัวได้แค่ไหน

ก็พวกมี backup ถึง backup เร็วแค่ไหนไร้นั้น ใช้ AWS Elastic Disaster Recovery(DRS)

และ อีก 2 ตัวเพิ่มเติม

1.Security

2.Cost

Business Continuity plan (BCP)- จะให้ทำงานยังไงตอนระเบิด

Recovery point objective (RPO) - อยากให้เสีย data มากสุดกี่นาที/ชั่วโมง

Recovery time objective (RTO) - อยากให้server ล่มกี่นาที/ชั่วโมง

สองอย่างนี้ trade off กัน

Disaster recovery choices (sort by Cost)

1.Backup and restore - backup แล้ว upload ขึ้นไปใหม่ ในตัวเดิม

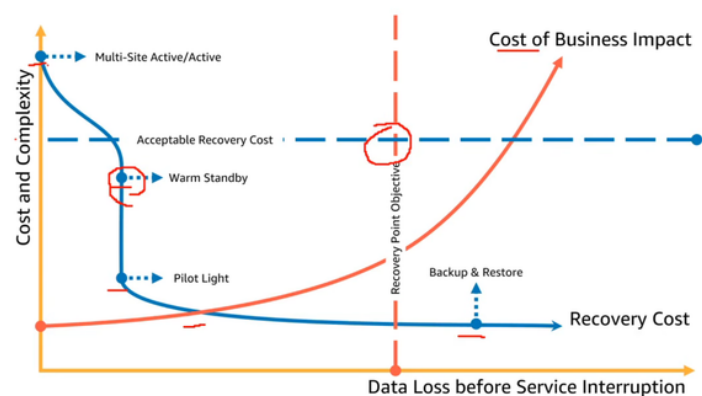
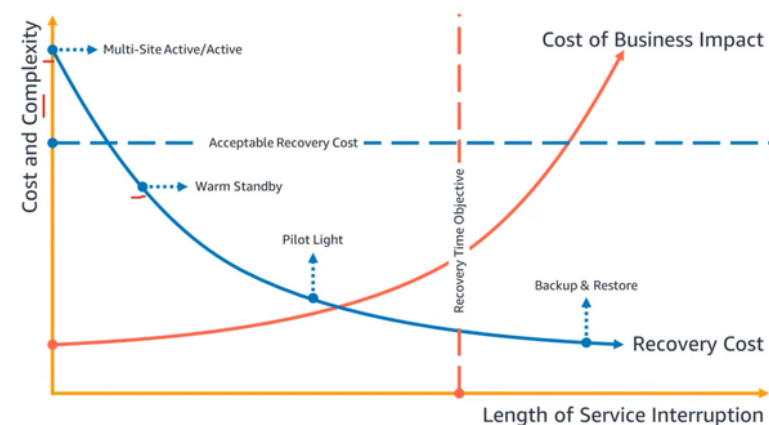
2.Pilot Light - sync DB แล้วไป run EC2 อีกโซน เปิดใหม่

3.Warm standby - sync DB มีหลาย server พังไปรันในอีกตัวที่ เล็กกว่าเปิดอยู่แล้ว

4.Multi-Site Active/active -sync DB เอาของก็อปไว้แล้ว ขึ้นไปในอีก โซนเลย ขนาดเท่า

กันเปิดอยู่แล้ว RTO

RPO



Application Programming Interface (API)- คือกฎที่ใช้บอกว่าเวลาจะทำอะไรกับฝั่ง server ที่มี API client จะต้องส่ง header + payload อะไรบ้าง (ฟ้a parameter function)

HTTP/HTTPS - เป็น protocol แ่ควบคุมวิธีการส่งหรือทำอะไรกับ อีกฝั่งเช่น ลบ ข้อมูล
AWS API - API ของ AWS

POSTMAN - ตัวช่วยเขียน FORMAT ในการใช้ API ของตัวๆนึ่ง จะมี parameter มาให้ และ ก็ไป config HTTP นิดหน่อยก็จะดึงมาได้และ

ปกติจะจัดการกับ API ด้วย AWS management console(ใช้interface web), AWS SDK(ใช้ภาษาโปรแกรม) , AWS CLI(ใช้พวกคำสั่ง shell)

Powershell - เป็น command line shell ที่ต่างจากตัวอื่นตรงที่ มันจะ return object .NET ซึ่งสามารถใช้ในการจัดการ AWS API ได้

Amazon Resource Names (ARNs) - เอาไว้ใช้ระบุว่าจะใช้อะไรของ AWS format มันคือ arn:partition:service:region:account-id:resource-type/resource_id

partition - ตามความเฉพาะของ top of region อารมณที่จีนแยกออกตัวหลักอะ services - ใช้service อะไร ex EC2 ,S3 ,IAM

Region - ใช้ AWS region ไหน

Account-id ของเรา

Resource_id - อาจเป็นเลขหรือ path

*ใช้ในการเอาทั้งหมดแบบไม่เลือกเฉพาะกลุ่มใดกลุ่มนึ่งไรจั้น ส่วนใหญ่ก็อปแปะ

::: อาจมีคั่นกลางกรณี global no region + account

.ใช้ gitpod กับ cloud9 ในการใช้พวก sdk ได้

Infrastructure as a code (IaC) - เขียน script ในการจัดการ automate ของ cloud infrastructure

AWS Cloudformation (CFN) - การเขียนแบบ explicit(ทุกอย่างต้องบอกอย่าง ละเอียด)ex.JSON,YAML

AWS Cloud Development kit (CDK) - การเขียนแบบ implicit(ใช้ของสำเร็จรูปไม่ได้ บอกตรงๆ) Python,JS,TS,C#,GO

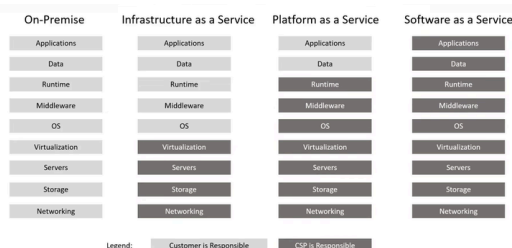
CDK ครอบ CFN ก็คือ CDK ถูกแปลงลง CFN อีกที

CDK - จัดการ Infrastructure SDK - ใช้งาน infra ที่ CDK สร้าง

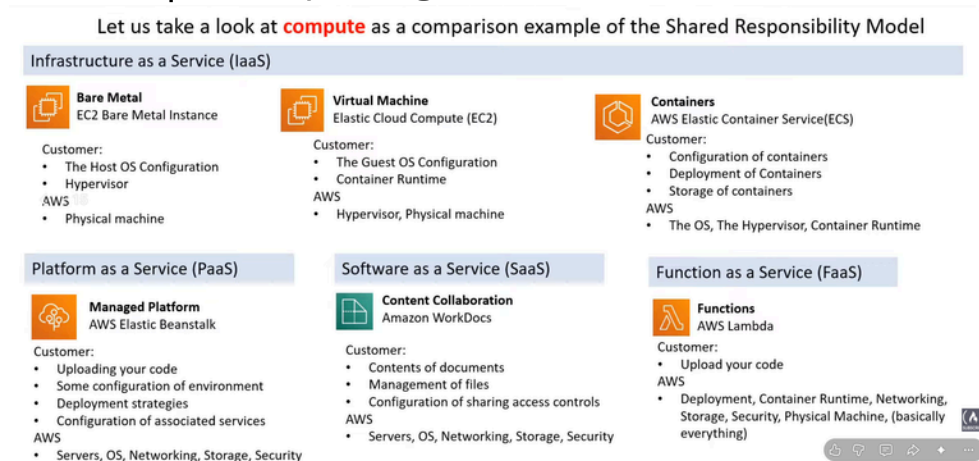
ใช้ AWS toolkit ผูกกับ VS ได้

Shared responsibility model - จะบอกประมาณว่า CSP รับผิดชอบส่วนไหน เราจะต้อง รับผิดชอบส่วนไหน แบ่งกันรับผิดชอบ มองง่ายๆคือ IN / OF the Cloud

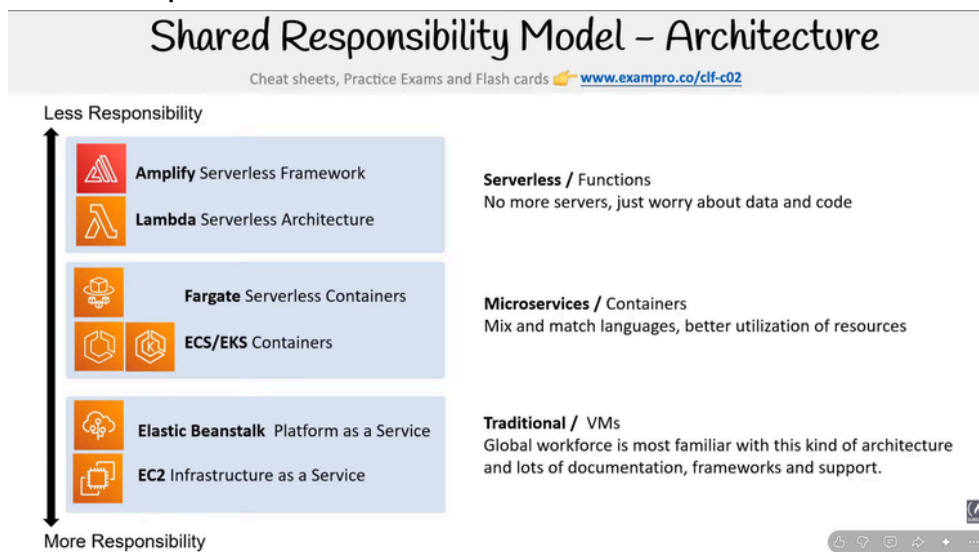
ในความต่างความรับผิดชอบในความต่างของ service คืออันที่เราต้องต้อง set นั้นแหละ



SRM ในยุ computing ex.



SRM ในยุ architecture ex



EC2 - สร้าง VM ได้ เรียกเป็น instance จัดการ os,cpu,ram,networks มีศัพท์เพิ่มเติม AMI (amazon machine image) - VM ที่ preconfig AWS services อื่นมันก็ on top EC2 เป็น backbone

Amazon Lightsail - EC2 but easier

Elastic Container Services (ECS) - EC2 instance but with docker

Elastic Container Registry(ECR) - เก็บ container images (อะไรที่แอปเราต้องใช้ในการรัน)

ECS Fargate - เป็น serverless ที่ง่ายตาม container ที่รัน ไม่มีไม่จ่ายถ้า ECS ต้องจ่ายค่า EC2 ถึงไม่ได้เปิด container

Elastic Kubernetes Services(EKS) - เอาไว้ใช้ K8s

AWS Lambda - serverless ส่งcode config mem function รันค้างเท่าไร ก็จะโดนตาม ที่มัน trigger เท่านั้น + 15 mins limit TLE

Higher Performance Computing services(HPCs) -HPC คือ เอาคอมหลายๆตัวมาช่วยคำนวณก็ supercomputer

The Nitro System - เน้นทำให้เร็วและปลอดภัย

มี Bottlerocket ที่ไว้ใช้รันLinux ที่เข้าใกล้ bare metal ได้

HPC - มี AWS Parallelcluster

Edge and Hybrid Computing services

Edge computing - เอา computing workload ไปอยู่ขอบใกล้ตัวเป้าหมายให้มากที่สุด

Hybrid Computing - เอา workload รันทั้ง datacenter เราและ cloud ของ CSP ก็พวก AWS outpost (physical rack), AWS Wavelength(เอาไปตั้งใน 5G), VMWare Cloud on AWS (ยก VMWare เราไปใส่ bare metal เขา), AWS local zones (ไม่ใช่ edge datacenters)

Cost and Capacity Management Computing Services

ต้องการลด Cost และ Capacity ต้องรองรับ traffic usage ได้

EC2 Spot Instances - แบบเช่าชั่วคราว ที่พร้อมโดนเจ้าของเตะออกเสมอถ้ามีแขก VIP มา ขอจ่ายราคาเต็ม

AWS Batch - ใช้วางแผนคำนวณ workloads

AWS Compute Optimizer - ลด costs และเพิ่ม performance โดย AI

ASGs - ย่อลดขนาด resource ที่ใช้ตามสถานการณ์

ELB - แบ่ง load ไม่ใช้โหนดทีเดียว (ลดการ vertical scaling)

EB - ลด operational cost โดยการไม่ set เอง ... เราแค่ไป deploy

Storage Services

Elastic Block File (EBS) - ให้ raw block มา ต้องนั่ง partition format เอง อาจใช้ FC, iSCSI

AWS Elastic File Storage (EFS) - มี file system มาให้ จะเชื่อมเรากับเขาด้วยการ mount ได้ โดยใช้ NFS (ระดับ OS)

Simple Storage Service (S3) - มีทุกอย่างให้เรียบร้อยแค่ ส่งแค่การจัดการข้อมูล พอ(ระดับ SaaS ใช้พอ)

S3 - เป็น object based storage not file based จะเก็บ S3 object ซึ่งอยู่ใน S3 bucket ตัว bucket ชื่อต้อง unique ฟ้า dir ชั้นเดียวกัน และที่ตัว object จะมี key(name object)-value(ข้อมูลของ obj) versionID(บอกversion obj)

metadata(ข้อมูลเพิ่มเติม obj) technically มันเก็บได้ infinitely but แต่ละตัวต้องตั้งแต่ 0 bytes - 5TB

level of S3 storage(เรียงตามความแพงมากไปน้อย)

1.S3 express one zone - access เร็วสุดๆ for ML or HPC

2.Standard S3 - ดึงข้อมูลเร็ว backup หลายที่

3.S3 Intelligent tiering - ก็มีการขยับข้อมูลบางส่วนไป level อื่นตาม ML คิด

4.S3-Standard IA - ก็คือ access ละเสียตั้งแต่เร็วอยู่ เน้นไม่ access

5.S3-One Zone IA - มี backup AZ ที่เดียวระเบิดแล้วไปเลย

6.S3 Glacier Instant Retrieval - access เร็วแต่ access แพง

7.S3 glacier Flexible Retrieval- access ที่นานไปนาทิจม แต่ถูก

8.S3 deep glacier - access ที่ 12 ชม

AWS Snow Family - ตัวใช้ส่งข้อมูลขึ้น cloud or ดึงลงมาจาก cloud หรือจะใช้ทำ Edge computing ก็ได้

Snowcone - มี 2 size 8TB(HDD) 14TB(SSD) เนื่องจากมันเล็กมันอาจเชื่อม network ส่งใน S3 โดยตรง(AWS DataSync)

Snowball edge - Storage Optimized(80TB), Compute optimized(39.5TB) โดยไอตัวนี้จะเอามาต่อกันได้เพื่อเพิ่มความหนาแน่น RAID

Snowmobile - 100PB storage

ส่งให้ S3 (ไอสองตัวหลังคือแบก data center ไปจริง) ซึ่งมันก็บังคับ Encryption และให้ hardware จัดไม่ได้

Storage gateway - เอา data center เราเชื่อมไปที่ cloud

File gateway - เชื่อมของเราเข้า S3 อาจเชื่อมด้วย NFS

Volume gateway - เอาของเราถือไปใส่ S3 ข้อมูลเราอยู่ที่เดิม แต่ backup ที่เขา(Stored volume) และย้ายข้อมูลไปอยู่ AWS และเอาตัวเองเป็น cache (Cached) ใช้ iSCSI

Tape gateway - เอาของเราไปเก็บไว้ใน tape เขา เพื่อ backup ระยะยาว

และก็มี AWS backup จัดการเรื่อง backup แลละ

มี amazon FSx เป็น File system เขา มี window(SMB) linux(Lustre)

Database - เอาไว้เก็บข้อมูล ที่ซับซ้อนเพราะออกแบบเยอะ มีการ optimize นู่นนี่นั่นเช่น query

Relational databases - พวกตาราง ใช้ sql ทั้งหมด

Non-relational databases - ก็อาจเป็นตารางหรือไม่ก็ได้จะนะ ex mongoDB

Data warehouse- ที่เก็บข้อมูลแบบ ตาราง (relational) ที่จะเน้น column เป็นหลัก (column oriented data-store) เอามา aggregate หรือคิดแบบ avg col นู่นนี่นั่นเน้นเร็ว

Key-value db - NoSQL (มันก็เป็น array แยกกันอะ แต่มันคล้ายตาราง) จับคู่ key value ตรงๆ ทำ aggregation, index, relationships ไม่ได้ เร็ว scale ง่าย

Document store - NoSQL ที่จะเก็บเป็น document อาจเป็น XML/JSON

NoSQL Database Service

DynamoDB - serverless key/value + document เน้นรับของใหญ่แล้วยังเร็ว ขยายง่ายมาก

DocumentDB - NoSQL ใช้ mongoDB ได้ ใช้ถ้าอยากใช้ mongoDB

Amazon Keyspaces - NoSQL KV ใช้เมื่ออยากใช้ Apache Cassandra

Relational DB services

Relational Database Service - ใช้ SQL

ก็จะมีหลายตัวใช้ได้เช่น MySQL , MariaDB , Postgres SQL , Oracle , MS SQL S และมีตัวพิเศษคือ Aurora - DB ที่จะถูกจัดการโดย MySQL or PostgreSQL แยกส่วน compute กับ storage ออกจากกันมี version serverless ด้วย

RDS on VMWare - เราเอา datacenter เราไปใช้ RDS (เหมือนปกติจะใกล้ๆ เล็กๆ)

Other DB services

Redshift - data warehouse เน้น query ซึ่ง

ElastiCache - เอา cache มาเพิ่มให้ db/server เรา DB เบื้องหลังที่เอามาทำ cache อาจเป็น Redis / Memcached

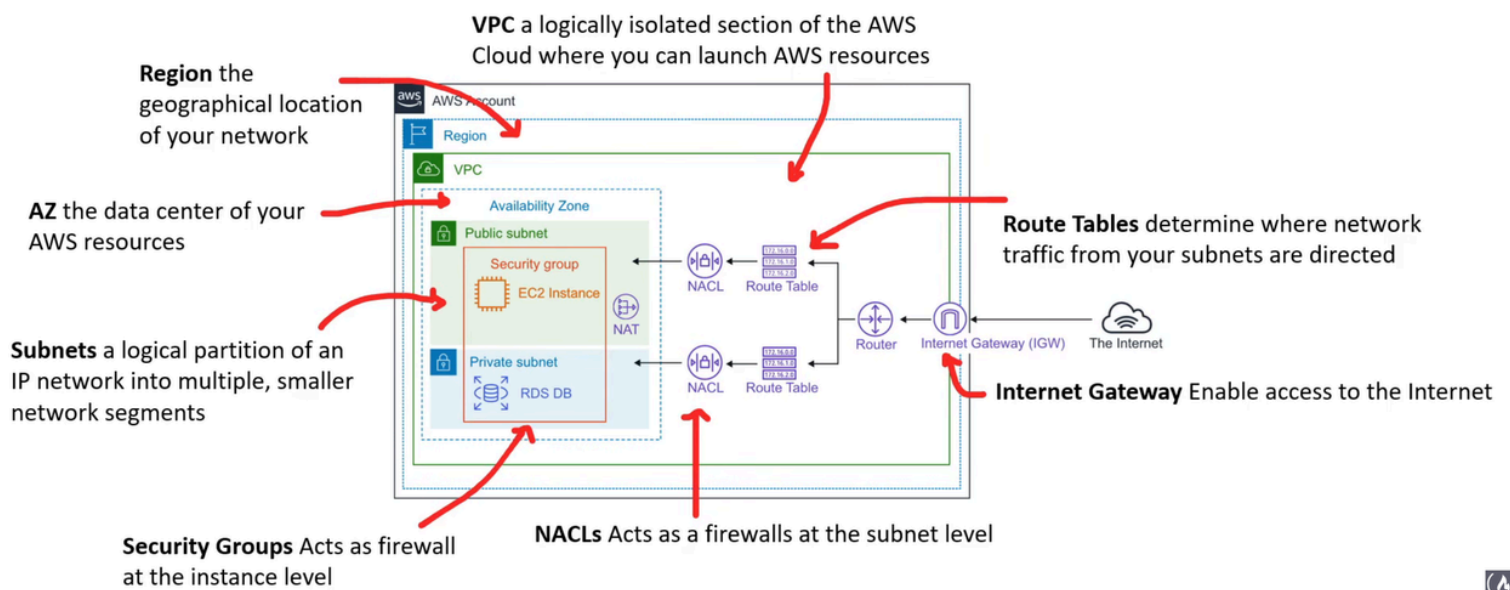
Neptune - graph DB

Amazon Timestreams - ใช้สำหรับ time-series DB

Amazon Quantum ledger DB - เข้าใจว่าใช้กับพวก transaction ที่ต้องการ trust สูงๆ จะไปอยู่วงการ finance

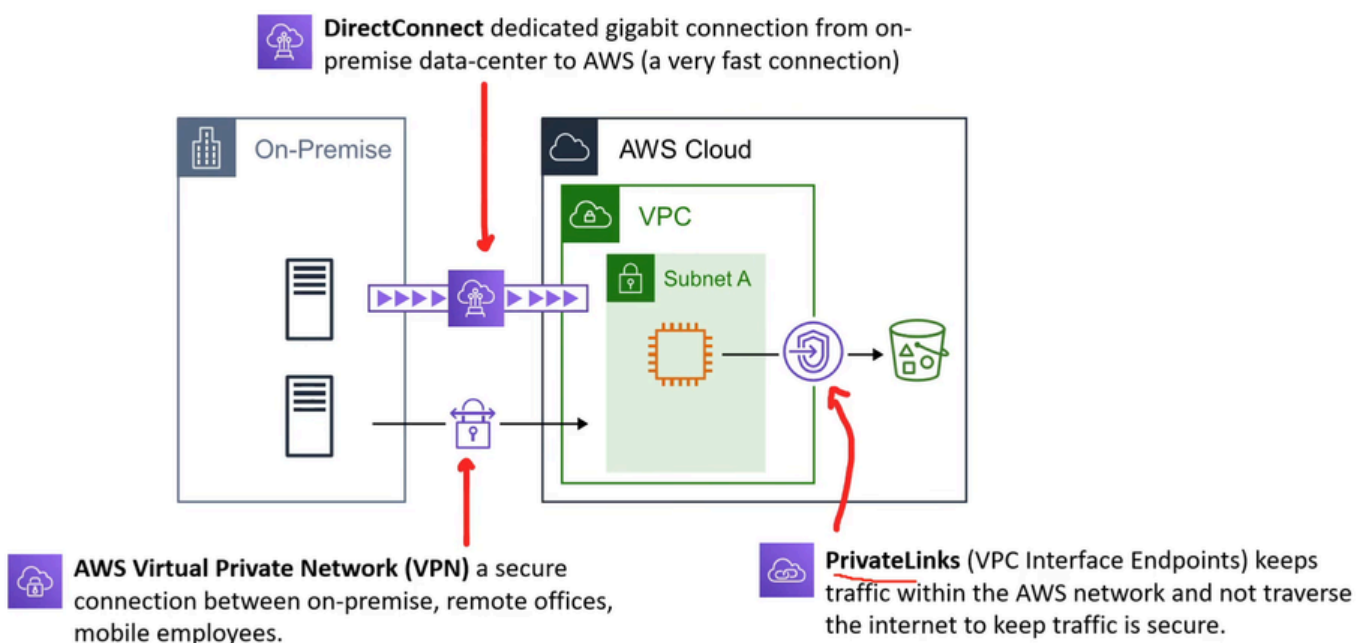
Database Migration Services (DMS)- บริการในการ migrate

Cloud Native - Networking services ตาม diagram เลย มี ตัวเชื่อมหาทางตัวกันไว้



VPC - isolated logical zone รับพวก resource AWS

Hybrid computing ex.



Virtual Private Cloud (VPC) and Subnets

VPC - isolated from the AWS network เซตรับ AWS resources เลือกอาณาเขตมัน โดย CIDR range (เอาจริงมันก็ subnet ตัวหลักปะ)

ก็จะมี subnet คือซอย portion ย่อยไปอีก เช่นเพิ่ม netmask ไรพวกนี้

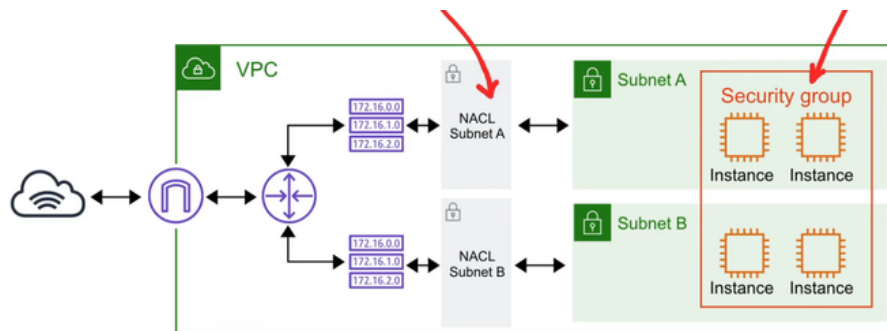
public access internet ได้ technically มันคือต่อกับ Internet gateway(IGW)

private can't access internet except using NAT gateway

มี route table ในการจัดการเส้นทางพวกนี้

Security groups - เป็น firewall ของตัว Instance เช่น EC2 โดย default มัน block หมดทุกคน feel ปรับ whitelisted ปรับได้แค่ allow rule ให้ใครเข้าได้บ้างการ block เรียง ตัวทำแบบจะเป็นไปไม่ได้ และเป็น stateful คือตั้งทีเดียวได้ทั้งเข้าทั้งออก

Network Access Control Lists (NACLs) - เป็น firewall at subnet level ก็สร้าง allow / deny rules ในการจัดการ IP ไล่แบบเรียงตัวไร้ และเป็น stateless หมายความว่า ต้องตั้งทั้งขาเข้าขาออก เพราะมันลืม



EC2 - virtual server that can be resizable

Choose OS → Instance type → storage → configure instance

Instance families - เอา combination พวก cpu mem etc มาจัดกลุ่มตามจุดประสงค์

1.general purpose - ใช้ทั่วไป

2.Compute optimized - เน้นสำหรับพวกที่ต้องการ processor แรงๆ

3.Memory Optimized - เน้นประมวลผลกับความจำใหญ่ๆ

4.Accerelated Optimized - พวกตัวเร่ง hw or co processorใช้กับ ML

5.Storage Optimized - I/O กับ storage โหด

EC2 Type → Size + Family ex.t2.micro มีตัวยกเว้น เป็น . metal ก็แบบ bare metal อะ

Level of tenancy

Dedicated Host - เอาทั้งตู้เลย สามารถ ดูข้อมูล พวก physical property ข้อมูล hardware เพื่อเอาไปจ่ายตังให้บอ software ไรนั้น

Dedicated Instance - จองตู้แต่ พอ reboot ก็จะมีไปตู้ที่ว่างเหมือนจองพื้นที่เฉยๆ

Default - reboot แล้วก็มีสุมที่ สุ่มตู้

ล็อก Ip โดยการ set elastic IP ได้

และก็สามารถเก็บ Image ตัว ec2 เก็บ vm เอาเอามาใส่อันใหม่ launch template โดยมันจะ เก็บอยู่ใน AMI amazon machine image

EC2 Pricing model

On-Demand - by default นั้นแหละจ่ายตามชั่วโมง อาจวินาทีก็ได้ (นาที่นึง charge หนึ่งที) เอาไว้ใช้ test อันใหม่เป็นระยะสั้นอะ

Reserved Instances(RI) - ใช้ของจองไว้เป็นปี หรือหลายปีถ้าเรารู้ว่ามันจะรันแน่ๆยาว ไม่ได้มีอะไรเปลี่ยน ราคาที่ลดจะมี factor ที่เกี่ยวข้องคือ Term(ยิ่งยาวยิ่งถูก), Class(instance ยึดหยุ่นน้อยกว่าถูกกว่า พวกแบบเปลี่ยน family instance ไม่ได้ตระกูล standard RI อีกพวกคือ Convertible RI) Payment Options(จ่ายเงินก้อนใหญ่ถูกกว่า)

RI attributes -เกี่ยวกับตัวส่วน class จะมีผลต่อราคาของ RI

1.Instance types

2.Region

3.Tenancy

4.Platform

ตัวนี้ไม่มีผลต่อราคาแต่มีต่อการใช้

Regional RI - ชื่อทั้ง region แต่ไม่ได้จองเอาไว้ ส่วนลด AZ ที่อยู่ใน region เลือก instance ใดใน family ก็ได้

Zonal RI - ชื่อแค่ AZ จะจองเอาไว้ ส่วนลดใน AZ ที่เลือกเท่านั้น ใช้ได้แค่ instance ที่กำหนดไว้แต่แรกเท่านั้น

regional ยึดหยุ่นกว่าแต่อาจไม่ได้ใช้แค่นั้นเมื่อเทียบกับ Zonal RI

มีเงื่อนไข limit ดังนี้ ต่อเดือนซื้อได้แค่ อย่างละ 20 instances + regional RI <= On-demand instance ที่มีอยู่ (ในแง่ cost นะ technically ทำได้แต่ทำทำไม)

Capacity reservation - บางทีเขาไปรัน instance ไม่ได้ hw มีจำกัดก็เลยให้แก้โดยการเลือกจองไว้ว่าจะใช้ตอนไหน แบบ request ไป เราก็จะเสียตังเท่าจำนวนตัว, ช่วงระยะเวลาที่จองไว้พวกนี้เหมือนคิด rate ondemand ปกติถ้ามี RI อาจจะลดได้บางส่วน (**ไม่เปิดก็โดนคิดอยู่ดีนะ**)

Standard RI - RI attribute modify ได้ ปรับขยาย instance ในรุ่นเดียวกันเปลี่ยน AZ แต่เปลี่ยน family ไม่ได้

Convertible RI - RI attribute จะเปลี่ยนได้คือ exchange อะเหมือนไป set ใหม่หมดเลย ซึ่ง Standard RI ลง marketplace ได้ แต่ convertible ลงไม่ได้

RI marketplace

-ขายได้เมื่อใช้ครบ 1 เดือน จำนวนเดือน วัน จะถูกตัดวันออก คือตั้งราคาเองได้แค่ส่วนที่ขายส่วนอื่นๆ เกี่ยวกับตัวข้างในราคาเดิม

Dedicated instance - โฉจองตู้แต่ตู้ชั่วคราวอะ แบ่งเป็น 3 ชั้นกดจองกันที่ จองไว้ล่วงหน้า(ลดราคา) และ onspot ตู้พร้อมโดนเตะ(ถูกสุด)

Savings Plan - ได้ส่วนลดแต่ต้องบอกว่าจะจ่ายเท่าไรรายชม สัญญาระยะยาว และมี 3 แบบ

1.compute plan - ใช้ EC2 , Fargate , Lambda ลดราคาหมด

2.EC2 Instance - ลดราคา instance นึงที่มาจาก family นึง + region

3.Sagemaker - ลดราคาการใช้ sagemaker มันเป็นพวกงาน ML/AI

Zero Trust model - Trust no one , Verify everything

ตอนแรกใช้แค่แบบพวก VPNs firewall

ตอนนี้ต้องใช้เพิ่ม คือการยืนยัน

AWS IAM (Identity and Access Management) – จัดการ perm จัดการผู้เข้าถึง เช็ทเวลาเข้า มันแค่เราต้องนั่งเช็ท มันอาจไม่ zero trust ทั้งหมด

AWS CloudTrail - Tracks all API calls

AWS GuardDuty - จับตัว SUS จาก cloudtrail or อื่นๆ

Amazon Detective - ใช้หา root cause หลังโดนไปแล้ว อาจเอาข้อมูลมาจาก guardduty ก็ได้

ก็อาจใช้ third party ที่ระบบมันฉลาดกว่าจะนะ เช่นของ Azure Active Directory , Google BeyondCorp , JumpCloud และก็ใช้ AWS Single Sign On

Directory Service - บริการชื่อ map กับที่อยู่ ex.DNS and Cloud Identity และตัว DB เก็บพวกข้อมูล identity ทั้งหมด ถ้ามองง่าย ๆ คือ DB ที่ทำได้หลายอย่าง

Active Directory - DB เก็บ User/password/device map subset Directory service เน้นด้าน identity verification

Identity Providers (IdP) - ตัวเก็บ user เรา และก็ตัว verify ให้เราใช้ไหม ex. OpenID OAuth 2.0, SAML

Single-Sign-On(SSO) - username + pw เดียวกัน ใช้ได้หลายอย่าง

Lightweight Directory Access Protocol(LDAP) - ใช้สื่อสารกับ active dir

Multi-Factor Authentication - ใช้ second device มายืนยันอีกรอบเวลา login

Security keys - เป็น device ที่สองที่ใช้ใช้งาน เช่นของ Yubikey

IAM

Policy - กฎผูก user (JSON)

Permission - อะไรทำได้ไม่ได้

Users - ตัวผู้ใช้ AWS

Groups - จัดกลุ่มเพื่อแบ่ง level perm

Roles - ให้ perm นั้นแหละ (ให้แบบชั่วคราว ไม่มีรหัสถาวร)

Principle of Least Privilege (PoLP) - ให้ perm user น้อยสุดเท่าที่จะยังทำงานรับได้

Just enough access(JEA) - ให้ perm เท่าที่จำเป็น

Just enough time(JET) - ให้เวลาใน perm เท่าที่จำเป็น

สองตัวนี้อาจใช้ ConsoleMe ได้ เพื่อบันทึกเข้าชั่วคราว

Risk-based adaptive policies - ดูความเสี่ยงการ access resource อาจใช้ third party

AWS Root-Users - full access account ลบไม่ได้ ต่างจาก user ปกติตรงลบไม่ได้กับเรื่อง perm ที่เรา set ได้ อาจใช้ service control policy(SCP) เพื่อบด perm ได้ root ใช้เปลี่ยน account setting , ปิด AWS account , เปลี่ยน plan

AWS IAM center - login แล้วใช้ resource ได้หมด

Application Integration - เอา 2 แอปเชื่อมกันโดยมีระบบตัวกลาง

Messaging System - ระบบส่งให้กันแบบ Async และมีการ decouple(ตัดการเชื่อมต่อ)

Queueing System - เป็น queue ถ้าไปอีกฝั่งแล้ว ต้องกด pull เพื่อส่งข้อมูลไปอีกฝั่งพอจบจะเคลียฝั่งส่งทั้งหมด ระหว่างนั้นอาจเก็บเพื่อส่งซ้ำ ex. Simple queueing Service (SQS)

Streaming - สามารถส่งให้หลาย consumer ได้แบบ real time ex. AWS Kinesis

Apache Kafka มีการเก็บข้อมูลไว้ระยะนึง

Publisher/Subscriber (Pub/Sub) - Pub ส่งให้ตัวกลาง event bus มันจะแบ่งกลุ่มแล้วก็ให้ subscriber มา subscribe พวกมันไว้ ถ้าอะไรมาลงที่ที่ subscribe ไว้ก็จะรับมาจาก

Publisher เป็นระบบ Fire-and-forget

ex. Real-time chat system , AWS Simple Notification System(SNS)

Amazon API gateway - ตัวไว้สร้าง API

State Machine - สถานะนึงไปสถานะนึงด้วยเงื่อนไขนึง ไว้ใช้ debug workflow ของเราได้ ex. AWS Step Functions

Event bus - รับ events แล้วก็เอา events นั้นส่งต่อให้ target ตามกฎที่ตั้งไว้ ex.

Eventbridge (serverless ใช้สำหรับmanage event มีให้ใช้หลายอัน event bus)

Application Integration Services

Amazon MQ - message broker โดย Apache ActiveMQ

Managed Kafka Service(MSK) - เอา kafka มาใช้

Appsync - ใช้ GraphQL (มีแบบ single API เรียกว่า GraphQL api กับใช้หลาย API มาทำให้เหมือนใช้อันเดียวเรียกว่า merged) เอาไว้ query จากหลายแหล่ง เราจะเขียน data model (schema) แล้วใช้ resolver ไปดึงข้อมูลตาม schema เรา ดึงได้หลายอย่างและมี cache แต่ต้องไปผูกกับ ec2 no serverless

AWS amplify - framework เพื่อให้ focus web dev / mobile app ทำงานกับ js framework ได้

Monolithic Architectures - รวมทุกอย่างไว้ที่เดียว

Microservices Architecture - ในหนึ่ง app อาจถูกแยกไปอยู่ในหลาย app แล้วมาประกอบกัน เป็น service เดิม

Kubernetes (K8s) - ใช้จัดการ container หลายๆตัว มันเหนือกว่า docker ตรง run containers ที่อยู่คนละ VM ได้ จะมี pods ซึ่งเป็นกลุ่ม container ที่ shared dependencies เดียวกัน และจะดีมากถ้าใช้สำหรับ microservice เพราะจัดการง่ายแต่ต้อง service เยอะๆ

Docker - Containerization Platform goal มันคือ pack software เป็น container

Open Container Initiative (OCI) - มาตรฐานด้านการทำ container แหะ

Podman - container ตาม OCI และก็เป็นตัวแทน docker มันไม่ใช่ daemon และสร้าง pods ได้แต่ต้องใช้คู่กับ Buildah-tool build OCI image Skopeo-ใช้ขยับ image ระหว่าง registry ต่างกัน

Services - ECS , Fargate , EKS , Lambda , EB, App Runner(manage container) , AWS Copilot CLI(build) , ECR , X-Ray(Debug microservices), Step Functions

Organization and accounts

AWS organizations - ตัวใหญ่สุด คลุมทุกอย่าง

Root account user - เป็นตัวที่ต้องมีใน AWS account มี perm สูงสุด ใน account นั้น

Organization Units - ใช้ AWS account(บัญชีcloud) ข้างในเป็นองค์กรย่อยอีกที ในองค์กรใหญ่

Service Control Policies(SCP) - เป็น policy คุมทุก user ในองค์กร ยกเว้นของmaster account

ข้างในเป็น AWS account → จะมี 1 root/master user แน่ๆ

ต่างกับ AWS User user เป็นแค่ตัวย่อยที่สร้างหลังจาก account พวก role อะ

AWS control tower - set AWS multi-account ง่ายปลอดภัยสำหรับองค์กร

Landing Zone - ของดีที่ถูกเช็ตไว้แล้วเอามาใช้

Account factory - เอาไว้สร้างจัดการ account อาจให้ perm ไปสร้างใน AWS service catalog

Controls - asset ของกฎให้เลือกใช้ได้แบบ pre config แล้ว

AWS config - เป็น compliance as a code (Cac)เพื่อ manage account per region
เข้าใจว่าเป็นตัวที่เราตั้งไว้แล้ว ถ้ามีคนปรับให้ไม่ตรงมันจะส่ง flag แจ้งเตือนว่าไม่ตรงกฎ

Tagging - key value pair เอาไปใส่ resource

Resource group - เอา tag มาจัดกลุ่ม

AWS Cloud Adoption Framework (CAF) - ใช้วางแผนการย้าย on prem → aws
จะโฟกัส 6 ด้าน

1.Business

2.People

3.Governance

4.Platform

5.Security

6.Operations

Business Centric Services

Amazon connect - เป็น call center ใช้เก็บข้อมูลสาย จัดการคิวสาย

Workspaces - ใช้เปิด virtual machine ที่set ทุกอย่างเรียบร้อยแล้ว linux window desktop

WorkDocs - แอร์ของโดยที่มีเจ้าของร่วมกัน

Chime - คล้าย zoom

WorkMail - เป็น gmail + calendar + contact

Pinpoint - ใช้ส่งเมลให้เป้าหมาย หลายคนพร้อมกันมั้ง แบบอาจแบ่งตามกลุ่ม

Simple Email Service (SES) - นักพัฒนาใช้ทำส่งเมลauto

QuickSight - เอาข้อมูลมารวมกันจากหลายแหล่งแล้ว plot graph

Provisioning service - การจอง/การสร้างให้ลูกค้า มักจะใช้เป็นบริบทแบบพวก high level จะไปสร้างหลังบ้านให้ลูกค้า อัตโนมัติ

EB - PaaS deploy web EB จะ provision AWS services เช่น EC2 ,S3 ,ASG etc... run dockerized env ได้ ไม่เหมาะกับบริษัทใหญ่

CloudFormation - IaC ใช้ provision ตัวอื่น

AWS Quickstarts(ปัจจุบันเรียกว่า AWS Partner Solutions) - packages ล่วงหน้าไว้ใช้ในทั้งระบบ เสกมาเลย

AWS Marketplace - ตลาดซื้อขาย software

AWS Amplify - serverless for mobile and web application provision พวก backends services ต่างๆ

AWS app runner - deploy containerized web/application ง่าย

AWS copilot - CLI บริหารจัดการ aws container service

AWS Codestar(EOL เรียบร้อยไป CodeCatalyst) - รวม UI ไว้ทีเดียว

AWS CDK - เขียน IaC ด้วยภาษาทั่วไป Project Management Dashboard

AWS Service Catalog - ก็คือเป็นศูนย์กลางที่สามารถให้คนอื่นมากดขอไปใช้ล่วงหน้าได้ ขอใช้ IaC ที่วางไว้แล้ว เอาไป launch

AWS SAM (Serverless Application Model) - ใช้เสก IaC ฝั่ง serverless

Serverless - แทนทุกอย่างเขาจัดการให้ เราแค่อัป code หรือข้อมูลเข้าไปและจ่ายเท่าที่ใช้ด้วยแบบ event ที่ใช้ ไม่ใช่ตามชั่วโมงทำให้จะถูกกว่า

DynamoDB - NoSQL key value / document based เน้น return ข้อมูลเร็วมากๆ

Simple Storage Service(S3) - เก็บข้อมูลแบบไม่จำกัดจำนวนตัว แต่ห้ามชื่อbucketซ้ำ

ECS Fargate - manage container + vm ให้ user แคใส่ container image เเฉๆ

AWS Lambda - deploy serverless

Step function - state machine ใช้นั่งเขียนการทำงาน และทดสอบเป็นโง

Aurora Serverless - DB ที่ออกแบบมาด้าน compute เฉพาะแต่เป็น serverless ยอม trade cold start กับใช้ไม่บ่อย เหมือนจะมี V2 ที่ตัด cold to start แต่ตัด Scale to 0

Serverless Architecture - มี scale แบ่งมัน แล้วแต่ CSPs จะนิยาม ฟีลลิ่งประหยัดไฟแต่ละเบอร์ ... อาจไม่จริงทุกข้อแต่เขาจะนับว่า serverless อะ

-ยืดหยุ่น ปลอดภัย ทำงานตลอดเวลา

-ทำให้ไม่ต้องสนใจ Infrastructure ด้านหลัง จ่ายเท่าที่ใช้จริงๆ

-Scale to zero ถ้าไม่มีใครใช้ ไม่ต้องจ่าย

Windows บน AWS - ใช้บน EC2 ,SQL Server or RDS,มีAWS Directory Service รับ Microsoft Active Directory (AD),AWS License Manager,FSx for window file server ,SDK(support .NET),Amazon Workspaces (Virtual Desktop) ,Lambda support Powershell

AWS Systems Manager (SSM) - Patch Manager update patch window

AWS Licensing Manager

หลักการ Bring Your Own License (BYOL) - เอา license ที่เรามีไปใช้เพื่อลด AWS มันคิดค่า software license ที่จ่ายด้วยถ้าเรามีก็จะลดได้

EC2 - ใช้ได้กับทุกแบบ tenant

RDS - Oracle DB

ถ้าเป็น MS SQL (ถ้าไม่มี software assurance) MwindowServer ต้องใช้ Dedicated host เท่านั้นหรือจองทั้งตู้แบบลือค

Logging services

CloudTrail - เก็บ log api call ทุกอัน ระหว่าง AWS service อยากเก็บนานๆ ต้องส่งไป s3 AWS Athena ช่วย query โดย SQL ได้

Cloudwatch - มี service ย่อยรวมกันเป็นมัน

-CW logs เก็บ log มาเป็น stream คล้ายๆ log มี timestamp ใน log จะมี ข้อมูล 1 event ใช้ logs insight ไว้ใช้ analyst event ได้นะ แล้วนั่ง query หาได้

-CW metric monitor ค่าตัวแปร ตามเวลา

-CW event(Eventbridge) - ถ้าเกิด event ก็ trigger อะไรสักอย่าง

-CW alarms - ตั้ง noti แล้วแต่จะตั้ง ใช้ trigger ได้เช่น ASG or EC2 recovery มี threshold condition

metric กับ data point (จุดข้อมูลใน metric)

Evaluation period ตรวจ zone นึงที่จุด

period จะให้เช็คทุกๆกี่นาที

Datapoint to alarm → ก็ถ้าเกินอันนี้แบบจะตั้งประมาณว่าเกิน 1 ใน 4 period ย้อนกลับไปมันจะ เต็งนะ

-CW dashboard - สร้างพวกกราฟตาม metrics

AWS X-ray - trace ดูว่าขยับยังงไปไหนพังที่ไหน debug หาตัวพัง